ForgeRAG: A Verifiable, Local-First, Multi-Plane Retrieval System for Engineering Reference Libraries

Rotoslider

Independent Researcher · github.com/Rotoslider/ForgeRAG · August 2026

Abstract. Retrieval-augmented generation is usually built on a single representation of a document — a bag of text chunks in a vector store — and evaluated on whether answers sound right. Neither choice survives contact with an engineering reference library, where the payload is a dimensioned drawing, a property table, or a code clause, and where a wrong answer costs real material. ForgeRAG is a retrieval system built for a private library of 494 engineering volumes (109,525 pages) that stores every page in three complementary planes — rendered page pixels with late-interaction visual embeddings, structurally-chunked text with dense and lexical indexes, and a typed knowledge graph of materials, processes, standards, and equipment extracted by a local language model — inside a single graph database, with the original page image as the terminal citation for every retrieval path. The system runs entirely on one desktop computer with one GPU; no external service receives a byte of the corpus. Two design commitments distinguish it from contemporaries. First, answers are generated by a vision-language model reading retrieved page *images*, not extracted text, so tables and diagrams survive the pipeline that usually destroys them. Second, and unusually, the store treats *verifiability* as a first-class property: a 28-invariant verification suite with exact counts, a single shared source of "work remaining" predicates, and repair operations proven to converge. This paper describes the architecture, the reasoning behind its deliberately unfashionable combination of thirty-year-old lexical search, mid-2000s graph modeling, and current late-interaction retrieval, and an evaluation across five search modes including a cross-book synthesis task and a vision-reading task. It closes with an honest accounting of where the design is worse than the dedicated vector databases and managed GraphRAG pipelines that currently dominate the field, and of what it costs to operate.

1 Introduction

The idea for this system predates the current retrieval-augmented generation (RAG) boom. In 2023 I wanted a single machine that could hold my working library — machine-design handbooks, ASM metallurgy volumes, welding codes, the NEC, ballistics texts, robotics papers — and answer questions from it the way a colleague with a full bookshelf would: by opening the right book to the right page, pointing at the table, and reading the number off it. Three constraints were fixed from the beginning and never negotiated away. The corpus is private and stays on my hardware. The answer must terminate in a page I can look at — a citation to a page image, not to an embedding's neighborhood. And the system must be able to tell me, with exact counts, whether its own index is complete, because a reference library that silently lost forty thousand pages is worse than no library at all. The third constraint sounded paranoid when I wrote it down. It turned out to be the most important one.

ForgeRAG is the result: an ingestion pipeline and retrieval stack, built with an AI pair-collaborator over an extended period, serving 494 documents and 109,525 pages from a FastAPI backend against a Neo4j store on one desktop. It is consumed by a web interface and, over a skills API, by autonomous agents that use it as their engineering reference. This paper is the design writeup I owed the system: what it is, why each piece is there, what it borrows from three decades of information retrieval, where it beats the current defaults, and where it loses to them.

2 Design Rationale: Old Ideas on Purpose

ForgeRAG is a deliberate mixture of old and new, and I want to defend the mixture rather than apologize for it. The fashionable RAG stack of the moment is a dedicated vector database holding dense embeddings of text

chunks, an approximate-nearest-neighbor index over them, and a language model that consumes the top-k chunks. Every element of that stack embodies an assumption that is false for reference books. Dense embeddings assume the query and the answer share semantics that a sentence encoder can capture; but the query "E7018" and the page that matters share only a token, and a 1990s lexical index — BM25 over an inverted file [1], here Apache Lucene inside Neo4j — finds it instantly where cosine similarity shrugs. Chunked text assumes the payload survives text extraction; but the payload of an engineering book is disproportionately tabular and graphical, and a stress–strain curve extracted to text is nothing at all. Top-k consumption assumes relevance is one-dimensional; but "which brass do I deep-draw for cartridge cases" is answered jointly by an alloy-properties volume and a metal-forming handbook, which no single similarity ranking interleaves reliably.

So the design reaches backward as much as forward. Lexical full-text search (mature by the late 1990s) is a first-class mode, not a fallback. A typed knowledge graph — materials, processes, standards, equipment, and the relations between them — is mid-2000s semantic-web thinking [2], rehabilitated because engineering entities really are discrete, named, and cross-referenced ("4340 is governed by an ASTM specification" is a fact with a stable shape, not a similarity). Reciprocal rank fusion [3], from 2009, does the mode-mixing with two lines of arithmetic and no learned weights. Onto this substrate the genuinely new pieces are grafted where they earn their place: late-interaction multi-vector page embeddings in the style of ColBERT [4] and ColPali [5] make page *images* retrievable; BGE-M3 [6] provides the dense text plane; a cross-encoder reranker [6] sharpens candidate lists; community detection via the Leiden algorithm [7] with LLM-written summaries, in the spirit of GraphRAG [8], gives corpus-level themes; and a vision-language model reads the final page images so that what was retrieved as pixels is also *answered* from pixels. Whether this eclecticism is a strength or a liability is a fair question; Section 8 takes it seriously. My claim is narrower than novelty: each component is the simplest tool that solves a real failure of the others, and the seams between them are where the engineering effort actually went.

3 The Storage Model: Three Planes, One Store, One Provenance

Every page in ForgeRAG exists in three planes. The **pixel plane** is the rendered page: a full-resolution PNG (300 dpi, retained on disk, 28 GB for the current corpus) and a reduced JPEG used for vision-model consumption and the web viewer. Each page also carries a late-interaction visual embedding — on the order of a few hundred token-level vectors after hierarchical pooling (a pooling factor of 3 reduces roughly 773 patch tokens to about 258 with no measured retrieval loss) — serialized as a binary property on the page node and scored at query time by the MaxSim operator [4]. The **text plane** is Docling-based [9] structural chunking: paragraphs, tables, figures, equations, and captions become typed chunks (279,044 of them) carrying page number, section path, and bounding box, each with an LLM-written summary, a 1024-dimensional BGE-M3 embedding over summary-plus-text, and membership in Lucene full-text indexes over both raw page text and chunk text. The **symbol plane** is the knowledge graph: an entity extractor prompts a locally-served LLM per page for typed mentions — materials with composition fields, standards with organization and code, processes, equipment, formulas, reference tables — which are merged into shared entity nodes connected to their mentioning pages; Leiden communities over the entity co-mention structure carry LLM-written summaries and their own embeddings.

All three planes live in one Neo4j database. This is an old-fashioned decision — the current idiom is one specialized store per representation, a vector database beside a document store beside a graph — and I made it for an old-fashioned reason: *synchronization is where retrieval systems rot.* When chunks, embeddings, entities, and page metadata are rows in four systems, "is the index complete?" is a distributed-systems question with no good answer. When they are all nodes and properties in one graph, completeness is a Cypher query. Neo4j's native vector indexes and Lucene full-text indexes are not the fastest of their kinds — Section 8 concedes this — but they are *views over the same store as the provenance*, and every retrieval mode terminates in the same `(:Document)-[:HAS_PAGE]->(:Page)` spine, which is the second commitment: whatever plane found the page, the citation is the page, and the page is an image you can open.

4 Ingestion, and What It Taught Me About Failure

Ingestion is a pipeline of renderer (PyMuPDF page rasterization with blank-page detection), visual embedder, Docling chunker with OCR for scanned volumes, summarizer, text embedder, and entity extractor, orchestrated as resumable jobs with a persistent step ledger, cooperative pause/stop, and a night-window scheduler so the GPU is free during working hours. The numbers that matter operationally: the local extractor (a 35B mixture-of-experts model with ~3B active parameters, served by LM Studio at roughly 135 tokens/second) costs 8–10 seconds per page of entity extraction, which makes the symbol plane, not the embeddings, the expensive asset — days of GPU time for a full library.

The honest history is that the first version of this pipeline was wrong in an instructive way: it converted failures into successes. A local LLM fails *constantly* — it truncates dense outputs, returns schema-valid empty results when confronted with a wall of tabular data, loops under constrained decoding, and times out under queue pressure — and early error handling quietly turned each of these into "page processed, nothing found." An audit eventually revealed roughly 38,000 pages the original passes had silently failed. The remediation produced what I now consider the system's most transferable ideas. Structured LLM calls read the generation's finish reason *before* parsing, escalate the token budget on truncation (8k→16k→32k) rather than re-prompting, and hand a typed truncation error to the caller, which splits the page text and merges half-extractions. A dense page whose extraction is empty is retried once with an explicit anti-bail instruction; an empty that survives is stamped *confirmed-empty*, a durable marker distinguishing "checked, genuinely nothing" from "never ran" — a distinction without which repair jobs re-pay the same pages forever. Text-less PDFs (vector-outline exports that look pristine but contain no extractable characters — a real class discovered via a manufacturer's manual with zero fonts in 52 pages) trigger a self-healing fallback that rebuilds a rasterized PDF from the already-rendered page images and lets OCR treat the book as scanned. Concurrent entity merges are serialized against page writes after a live race destroyed relationships seconds after they were written. None of this is glamorous. All of it is the difference between an index and an index-shaped liability.

5 Retrieval

Five user-facing modes sit over the three planes. **Keyword** is Lucene over page text with a phrase-boosted query, metacharacter escaping (engineering queries are full of "3/16" and "weld(ing)"), and optional edit-distance fuzzing for OCR-era typos, with an all-terms conjunction boost so that a typo'd "weldlng electrode" ranks welding-electrode pages above pages merely dense in "electrode." **Semantic** is dense BGE-M3 retrieval over pages. **Visual** is MaxSim over the multi-vector page embeddings — the mode that finds "stress strain curve diagram" as a picture. **Chunk** retrieval fuses dense and BM25 evidence at chunk granularity and reranks with a cross-encoder, for precise quoting. **Hybrid** is the workhorse: reciprocal rank fusion over three retrievers (chunk-dense, chunk-BM25, page-dense) with reranking, plus graph-aware strategies — *graph-boosted*, which folds entity-mention evidence into the fused ranking, and *graph-first*, which resolves query terms to entity nodes and follows mention edges to pages, falling back to fusion when no entities resolve. Entity resolution itself is served by an in-memory matcher over all 276,487 entity name-forms; an early linear implementation was quietly useless at this scale (its time budget covered 0.007% of the population), and its replacement — an exact normalized-name table plus a character-trigram inverted index generating candidates for edit-distance scoring — answers in 31–48 ms with full coverage, which is what makes the graph strategies actually graph-aware.

Answer mode is the synthesis path and the system's reason to exist. Retrieval (fusion by default, degrading to keyword when embedding services are down) selects pages under a per-document diversity cap, so a question whose answer spans books gets more than one book in context; graph exploration seeded by the query and the retrieved pages contributes reasoning chains ("material → governed-by → standard") and reserves context slots for graph-discovered pages. The selected pages are then sent to a vision-language model *as images* — with adjacent pages included for tables that run across page boundaries — under a citation protocol in which each

image carries an ID the model must cite, decoupling citations from the books' own printed page numbers. The model reads pixels: property tables, curve axes, schematic component values. This is markedly more expensive than text-stuffing (16–24 seconds per answer on local hardware) and markedly more truthful about documents whose truth lives in figures.

A sixth lane was added after this paper's first evaluation, closing a weakness Section 8 originally had to concede: **summary search** over hierarchical section summaries in the manner of RAPTOR [15], but guided by document structure rather than embedding-space clustering. RAPTOR clusters chunks because most corpora carry no structure; reference books do, and Docling has already stamped every chunk with its heading path. The builder therefore rolls chunk summaries (figure and table chunks included) up into leaf-section summaries, sections into chapters, chapters into a whole-document summary — each level written by the local LLM, embedded, and stored as first-class nodes with page ranges, with page-window fallback sections for scanned books without headings. Zoom-out questions ("what does this volume cover"; "which sections treat radar odometry") retrieve at the appropriate abstraction level and cite the section's opening page. In staged validation the lane resolved section-level queries exactly (a drift-characteristics query returned §9.2, §9.2.3, §9.2.1 of a robotics handbook in order) and whole-document queries to the correct root summaries; its evaluation is preliminary relative to the audited modes of Section 7, and the plane inherits the same verification-and-drain machinery as the others from birth.

6 Verifiability as a First-Class Property

The feature I would defend longest in front of a hostile committee is not a retrieval trick. It is that ForgeRAG can prove the state of its own index. A verification suite checks 28 invariants with exact counts and no sampling: page counts and numbering contiguity per document, orphan detection, on-disk image existence, text-count consistency, embedding dimensionality and blob byte-integrity, chunk existence and well-formedness *per document* (existence checks matter — a document with zero chunks passes every "all chunks are valid" check vacuously, a blind spot found the hard way), entity-extraction coverage including the confirmed-empty protocol, community coverage, index health including vector-index *dimension* configuration (a stale-dimension index stays online and silently indexes nothing after a model change), and agreement between the audit's arithmetic and the repair system's selectors. That last check embodies the governing rule, learned from the worst class of bug this project produced: *the definition of "work remaining" lives in exactly one module*, imported by the audit that reports gaps, the drains that repair them, and the verifier that cross-checks both — because when those definitions drift, a repair can run, report success, and change nothing the audit sees. Every repair operation is designed to converge: re-runs select only missing work, markers make terminal states durable, failures leave loud unfinished states rather than quiet finished-looking ones. The operational payoff is a one-screen answer to "is my library whole?" — currently: every one of 109,525 pages reachable by at least one retrieval plane, all text pages either entity-extracted or confirmed-empty, two pages (0.002%) in a visible, explained failure state.

7 Evaluation

The evaluation is an audit rather than a benchmark: a 20-document, five-tier question set spanning exact codes to vague natural questions to cross-book synthesis, with a page-grounded answer key built by direct corpus inspection, executed three ways — a 45-test scripted battery against the API; an independent gauntlet run by an autonomous agent over the skills interface (a genuinely independent tester: it found integration bugs the direct battery structurally could not, including a tool-routing fault in its own client and an off-by-truncation bug that had excluded graph-discovered pages from every answer ever generated); and a vision tier in which the agent retrieved page images and answered from pixels alone.

Table 1. Audit results by mode (page-grounded answer key; "hit" = expected source in top 3).

Mode	Tests	Result	Typical latency

Mode	Tests	Result	Typical latency
Keyword (exact codes)	5 + 3 edge probes	5/5, all probes clean	<0.1 s
Semantic (phrases)	5	5/5	0.1–0.6 s
Visual (diagram queries)	3	3/3	0.1–5 s
Hybrid + graph strategies (vague)	9	9/9 topical; graph-first weak on entity-free queries until fallback added	0.1–0.7 s
Answer (cross-book synthesis)	5	5/5; up to five distinct documents cited per answer	16–24 s
Knowledge graph (typed queries)	2 + bonuses	Exact expected pages; correct ASTM/AMS specs	<0.3 s
Vision (read retrieved pages)	5	5/5, incl. exact table values (36 ksi; 30% Zn)	per-image VLM cost

Two results deserve prose. The cross-book task asked for the composition of AISI 4340 *and* the preheat reasoning for welding it; the system's answer drew composition from a metals desk reference and preheat/cooling-rate discussion from a welding handbook — five documents, correct chemistry, correct metallurgy — which is precisely the multi-volume behavior a single-representation retriever fails to produce. In the vision tier, the reading agent twice *declined* to report values that were not printed on the page it had opened (a copper percentage present only as a balance-by-footnote; a tensile range on a page the search had not returned), deriving what was derivable and refusing the rest. A retrieval system that supports that behavior — where the model's evidentiary horizon is the retrieved pixels — is the design goal made visible.

8 Against the Field: Where This Wins and Where It Loses

Compared with the dedicated vector-database stack (FAISS/Milvus/Qdrant/pgvector-style, with chunked text and a text-only LLM), ForgeRAG wins on provenance (citations terminate in page images), on tabular and graphical content (the pixel plane and VLM reading avoid the text-extraction bottleneck entirely), on exact-identifier queries (lexical search as a peer mode), on multi-book synthesis (fusion plus diversity caps plus graph context), and on auditability (none of the popular stores can answer "what fraction of my corpus is actually indexed, and prove it"). It loses on raw vector performance and scale: Neo4j's HNSW indexes are adequate at 10^5 pages but a purpose-built ANN store is faster, more memory-frugal, and horizontally scalable where this design is deliberately single-node. It loses on ingestion cost: a pure-embedding pipeline indexes a book in minutes; ForgeRAG's symbol plane spends days of local LLM time, and that plane's quality is chained to the local model's — my graph carries measurable noise (generic-noun entities, occasional hallucinated relations) from earlier extractor generations, requiring cleanup machinery a vector store never needs. It also loses on operational simplicity: three planes and their invariants are exactly three times the surface of one.

Compared with GraphRAG-style pipelines [8], the differences are philosophical. GraphRAG builds its graph to *summarize* — communities and their reports are the retrieval substrate for global questions — whereas ForgeRAG's graph exists to *index and cross-reference*: typed entities with page-level mention provenance, communities as a supplementary lens. ForgeRAG keeps every hop inspectable down to a page image, where summarization-first designs answer from prose about prose. An earlier draft of this paper conceded here that ForgeRAG was weaker at zoom-out questions than systems built around hierarchical summaries; the structure-guided summary plane of Section 5 was built in response, and the concession narrows to this: cross-corpus thematic questions still lean on entity communities, which cut across books by co-mention rather than by argument, and the summary plane's evaluation is younger than the rest of the system's. Compared with the

managed cloud RAG products appearing monthly, the comparison is short: they are faster to stand up, scale further, and are excluded here by the first constraint — the corpus does not leave the building. The three-year-old bet, that a private, verifiable, page-faithful system would be worth the operational weight, reads better in 2026 than it did in 2023: models capable of reading a page image and extracting typed entities now run on one workstation GPU, which is the capability this design was waiting for.

9 What It Takes to Run

The reference deployment is one desktop: a small-form-factor PC hosting the services, with a workstation GPU (48 GB class; the pipeline degrades gracefully to smaller cards with smaller batch sizes) for embedding models and the local LLM/VLM. Software: Python 3.12/FastAPI backend; Neo4j 5.19+ (vector indexes and full-text indexes required); LM Studio or any OpenAI-compatible server for the extractor/answerer (currently a 35B-A3B mixture-of-experts model with structured-output support); BGE-M3 and bge-reranker-v2-m3; a ColPali-class visual embedder; Docling for structural parsing; a React web client. Storage for the current corpus: ~28 GB page images, a Neo4j store in the tens of GB, plus staged PDFs. Costs to expect: ingestion at 8–10 s/page of LLM extraction (a 1,000-page handbook is a night, a full library is days — the scheduler exists precisely for this); answers at 16–24 s; every other mode interactive at well under a second. The system assumes a trusted single-operator network: there is no authentication layer, and filesystem-touching endpoints are path-confined rather than access-controlled. That is a stated scope, not an oversight, but it bears repeating.

10 Limitations

Beyond the comparative losses above: extraction depth varies by ingestion era, because the extractor's prompt, schema, and underlying model improved over the project's life — earlier books carry shallower, noisier symbol planes than recent ones (sampling puts the older layer at parity on named designations but padded with generic-noun entities). Keyword ranking is term-frequency driven and sometimes surfaces a book's index page above its data page; the practical remedy (fetch the adjacent candidate too) works but is a workaround. Graph-first retrieval is only as good as entity extraction, and inherits its noise. The single-store bet caps corpus growth at what one Neo4j instance and one GPU can serve; I estimate comfortable headroom to the mid-10^5 page range on current hardware, not beyond. Model migration (changing embedding dimensions) is a re-embed of the affected plane, detected but not automated. And the evaluation, while adversarially constructed and independently executed, is a 20-document audit with a hand-built key, not a public benchmark; its numbers claim health, not state of the art.

11 Conclusion

ForgeRAG argues, by existing, that the current defaults in retrieval-augmented systems are choices rather than laws. Text chunks in a vector store are one representation among three that a reference library needs; a dedicated vector database is one storage answer, and keeping provenance, symbols, and vectors in a single graph is another with different virtues; and "the answer sounded right" is a much weaker property than "the index is provably complete and the citation is a page you can open." The system's parts are individually old — inverted files, typed graphs, rank fusion — or borrowed from current literature — late interaction, dense multilinguality, community summaries, vision-language reading. The contribution is the joinery, and the operational doctrine that failure must be loud, repairs must converge, and completeness must be a query. Three years after the idea and one long collaboration later, the library answers from its own pages, refuses what its pages do not say, and can prove what it holds. That was the specification all along.

Acknowledgments

The concept, corpus, requirements, and relentless field-testing are the author's; the implementation was developed in an extended collaboration with Anthropic's Claude, which co-wrote the codebase and this paper's drafts under

the author's direction. The system's autonomous agent users ("Chooms") served as unwitting and then witting test pilots, and found bugs their creator could not.

References

[1] S. E. Robertson and K. Spärck Jones, "Relevance weighting of search terms," *J. Am. Soc. Inf. Sci.*, 27(3):129–146, 1976; and S. E. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, 3(4):333–389, 2009.

[2] T. Berners-Lee, J. Hendler, and O. Lassila, "The Semantic Web," *Scientific American*, 284(5):34–43, 2001.

[3] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. SIGIR*, 2009, pp. 758–759.

[4] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. SIGIR*, 2020, pp. 39–48.

[5] M. Faysse, H. Sibille, T. Wu, B. Omrani, G. Viaud, C. Hudelot, and P. Colombo, "ColPali: Efficient document retrieval with vision language models," arXiv:2407.01449, 2024.

[6] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," arXiv:2402.03216, 2024.

[7] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," *Scientific Reports*, 9:5233, 2019.

[8] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024.

[9] C. Auer et al., "Docling technical report," arXiv:2408.09869, 2024.

[10] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[11] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.

[12] Neo4j, Inc., "Neo4j graph database: vector search indexes and full-text indexes," product documentation, 2025. Apache Lucene, lucene.apache.org.

[13] Qwen Team, Alibaba Group, "Qwen technical reports," arXiv:2309.16609 and successors, 2023–2026.

[14] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, 2017.

[15] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive abstractive processing for tree-organized retrieval," in *Proc. ICLR*, 2024. arXiv:2401.18059.

Source, issue history, and the verification suite described in §6 are available at github.com/Rotoslider/ForgeRAG. The evaluation harness and answer keys are archived with the repository. This document was prepared August 2026.